

Complex Computing Problem (CCP)
Course Code & Title: CSC202 Artificial Intelligence

1. Course Learning Outcome

CLO 3: Perform informed, uninformed search, adversarial and local search strategy algorithms to solve goal-oriented problems.
Level of CLO: CLO3 is mapped to the 5th level, i.e., Evaluate of Bloom's taxonomy.

2. CCP Attributes

- **A2 Depth of Analysis Required:** Has no obvious solution and requires conceptual thinking and innovative analysis to formulate suitable abstract models.
- **A3 Depth of Knowledge Required:** A solution requires the use of in-depth computing or domain knowledge and an analytical approach that is based on well-founded principles.
- **A8 Interdependence:** Is a high-level problem possibly including many component parts or sub-problems.

3. Problem Statement

Design and develop an AI-powered Multi-Level Sudoku Solver and Evaluator that applies and compares multiple search and constraint-based algorithms to solve Sudoku puzzles of varying difficulty. The system must demonstrate, evaluate, and contrast the performance of at least four distinct AI search strategies, providing both a solution and a measurable analysis of each algorithm's efficiency.

Background

Sudoku is a classic Constraint Satisfaction Problem (CSP) that serves as an ideal testbed for AI search strategies. A standard 9x9 Sudoku grid must be filled with digits 1–9 such that every row, column, and 3x3 sub-grid contains each digit exactly once. The problem scales in complexity with difficulty level, making it suitable for evaluating and comparing the practical effectiveness of different search paradigms.

Students are required to implement a levelled puzzle system: Easy, Medium, Hard, and Expert, where higher difficulty levels have fewer pre-filled cells, higher search depth, and greater branching factor, thereby stressing each algorithm differently and producing meaningful comparative data.

Problem Description

The application must support the following core features:

1. Multi-level Sudoku puzzle generation supporting at least four difficulty levels (Easy, Medium, Hard, Expert), with difficulty determined by the number of pre-filled cells and puzzle uniqueness.
2. Implementation of Uninformed Search: apply Backtracking Search (depth-first) as the baseline uninformed strategy, demonstrating exhaustive state-space exploration without domain heuristics.
3. Implementation of Informed Search: apply the Arc Consistency (AC-3) algorithm combined with the Minimum Remaining Values (MRV) heuristic and Degree Heuristic to prune the search space intelligently before and during solving.
4. Implementation of Local Search: apply a Simulated Annealing or Hill Climbing approach, starting from a complete (but possibly inconsistent) assignment and iteratively reducing constraint violations.
5. Implementation of a Constraint Propagation strategy as a standalone solver using Forward Checking with constraint propagation to eliminate invalid candidates at each step.
6. Algorithm Performance Evaluation: for each puzzle and algorithm, record and display metrics including time taken to solve, number of states explored, number of backtracks performed, and solution optimality.
7. A comparative analysis dashboard (CLI table or GUI chart) that visualises performance differences across all four algorithms at each difficulty level.

8. An optional adversarial or competitive mode where two AI agents race to solve the same puzzle using different strategies, and the system evaluates which agent performed better and why.

The system must be developed in Python. Students are encouraged to design the application in a modular and extensible manner, separating puzzle generation, algorithm implementation, constraint handling, and performance reporting into independent modules, similar in structure to professional AI benchmarking frameworks.

4. Instructions

- You may work in a group of three members.
- Your project will be graded according to the provided rubrics.
- Deadline to submit: June 7, 2026

5. Deliverables

Students are required to submit the following:

1. Source Code

- Complete and well-documented Python code with clearly separated modules for: puzzle generation, each search algorithm, constraint handling, and performance reporting.

2. Project Report

- Introduction and problem overview
- Problem formulation as a CSP (state space, variables, domains, constraints)
- Algorithm design and justification (why each search strategy is appropriate)
- Implementation details (backtracking, heuristics, propagation, local search)
- Performance analysis with tables and graphs comparing all algorithms across difficulty levels
- Challenges faced and solutions applied

3. User Interface (UI)

- Functional CLI or GUI that allows users to: select difficulty, choose algorithm, view the solving process step-by-step, and display the performance comparison dashboard.

4. Demonstration / Presentation

- Live demo showing all four algorithms solving puzzles at each difficulty level.
- Walkthrough of performance comparison results with discussion of findings.

5. Testing Evidence

- Screenshots or logs showing each algorithm solving puzzles at all four difficulty levels.
- Performance data tables showing measurable differences (time, states, backtracks).

6. Alignment with Attributes of Complex Computing Problems

Attribute	Alignment / Justification
Depth of Analysis Required	Students must formally model Sudoku as a CSP, analyse the trade-offs between uninformed, informed, and local search strategies, and design experiments to evaluate algorithm performance. No single algorithm is universally optimal; students must think critically about heuristic choices, constraint propagation depth, and the impact of puzzle difficulty on algorithm behaviour.
Depth of Knowledge Required	Requires in-depth understanding of search algorithms (BFS, DFS, backtracking, A*, hill climbing, simulated annealing), constraint satisfaction principles (arc consistency, MRV,

	forward checking), and their theoretical complexity properties. Students must apply well-founded AI principles to implement, tune, and justify each algorithm's design choices.
Interdependence	The system consists of multiple interdependent components: the puzzle generator, the CSP formulation layer, each search algorithm module, the constraint propagation engine, and the performance benchmarking dashboard. Each component must interface correctly with others; a change in the constraint model or difficulty level directly affects all algorithm modules and performance metrics.

7. Complex Computing Problem Assessment Rubrics

Criteria (Marks)	Description	Excellent	Good	Satisfactory	Needs Improvement
Search Algorithm Implementation (3 Marks)	Correct implementation of all four required search strategies: uninformed (backtracking), informed (AC-3 + MRV), local search, and forward checking.	All four algorithms correctly implemented and producing valid solutions.	Three algorithms work correctly; one has minor issues.	Two algorithms work correctly; others are incomplete.	Fewer than two algorithms implemented or all produce incorrect results.
CSP Formulation & Constraint Handling (2 Marks)	Correct formal modelling of Sudoku as a CSP with proper variable, domain, and constraint definitions, and valid constraint propagation logic.	CSP fully and correctly modelled; all constraints enforced throughout solving.	CSP mostly correct with minor constraint violations.	Partial CSP formulation; some constraints missing or incorrectly applied.	No formal CSP formulation or constraints not enforced.
Performance Evaluation & Comparison (2 Marks)	Collection and presentation of performance metrics (time, states explored, backtracks) across all four algorithms and all difficulty levels, with meaningful comparative analysis.	Comprehensive metrics collected; clear, insightful comparative analysis provided.	Metrics collected for most algorithms; basic comparison present.	Limited metrics; comparison is superficial or incomplete.	No performance evaluation or comparison attempted.
Puzzle Generation & Difficulty Levels (1 Mark)	Generation of valid, uniquely-solvable Sudoku puzzles across at least four difficulty levels.	All four difficulty levels generate valid, unique puzzles consistently.	Most difficulty levels work; minor generation issues.	Only one or two difficulty levels functional.	Puzzle generation not implemented or produces invalid puzzles.
Code Structure & Documentation (1 Mark)	Modular code design with clearly separated components and inline comments explaining algorithmic logic.	Clean, modular, and well-documented code across all modules.	Mostly organized with some comments.	Understandable but lacks modular structure.	Poorly structured or undocumented code.
Optional / Advanced Feature (1 Mark)	Implementation of the adversarial competitive mode, a GUI visualiser showing the step-by-step solving process, or a 16x16 (Hexadoku) variant.	One or more advanced features implemented effectively.	Attempted but incomplete.	Feature mentioned but not functional.	No attempt made.